Fast Computation of Multivariate Subresultants

A report for the CCA Project #13

Yunus GÜRLEK

Frédéric XIA

Supervisor:

Weijia WANG

May 21, 2026

Contents

1 Introduction **2**

2 Preliminaries **3**
2.1 Resultant 3
2.2 Subresultant polynomials 4

3 Reduction properties **6**
3.1 Specialization property of the resultant 6
3.2 Specialization property of subresultant polynomials 7

4 Proposed Algorithm **8**

5 Benchmarks **9**
5.1 FMPZ Benchmarks 9
5.2 FMPQ Benchmarks 11

6 Conclusion **13**

1 Introduction

Let R be a ring, and let $f, g \in R[x]$ be two non-zero univariate polynomials of degree p and q, respectively, with $p > q$. When R is a domain, we denote by $\mathbb{K}$ its fraction field, and by $\gcd(f, g)$ the monic greatest common divisor of f and g in $\mathbb{K}[x]$.

To compute $\gcd(f, g)$, a common approach is to use the Euclidean algorithm, which consists in computing the sequence of remainders $r_0 = f$, $r_1 = g$, and $r_{i+1} = r_{i-1} \mod r_i$ until we reach a remainder that is zero. The last non-zero remainder is then the greatest common divisor of f and g. However, this algorithm can be inefficient, as the size of the coefficients of the remainders can grow rapidly, leading to a large number of bit operations.

Example 1. *Let* $f = x^8+x^6-3x^4-3x^3+8x^2+2x-5$ *and* $g = 3x^6+5x^4-4x^2-9x+21$. *Applying the Euclidean algorithm with* $r_0 = f$, $r_1 = g$, *we have the following remainder sequence:*

$$r_2 = -\frac{5}{9}x^4 + \frac{1}{9}x^2 - \frac{1}{3}, \qquad r_3 = -\frac{117}{25}x^2 - 9x + \frac{441}{25}$$
$$r_4 = \frac{233150}{19773}x - \frac{102500}{6591}, \qquad r_5 = -\frac{1288744821}{543589225}$$

Thus, we have $\gcd(f, g) = 1$.

The subresultant theory, developed in the 19th century, offers a more efficient method for computing $\gcd(f, g)$. The subresultant polynomials $(\mathrm{sResP}_i(f, g))_{0 \le i \le q}$, defined over $R[x]$, have the key property that an appropriately chosen subsequence is proportional to the remainder sequence produced by the Euclidean algorithm, as can be seen in the following example.

Example 2 (continued)**.** *In the previous example, the subresultant polynomials are given by:*

$$\mathrm{sResP}_5(f, g) = 15x^4 - 3x^2 + 9, \qquad \mathrm{sResP}_3(f, g) = 65x^2 + 125x - 245,$$
$$\mathrm{sResP}_2(f, g) = 9326x - 12300, \qquad \mathrm{sResP}_0(f, g) = 260708.$$

Indeed, the bit size of the coefficients is better controlled, and we still have $\gcd(f, g) = 1$.

The relation between the subresultant polynomials and the remainder sequence of the Euclidean algorithm was already known to Euler [7], while its use in the context of computer algebra was first proposed by Collins in 1967 [4]. Since then, subresultant-based algorithms have been widely studied and implemented in computer algebra systems and libraries, such as Wolfram, Maple, and FLINT.

State of the Art. Subresultant polynomials can be computed through various methods, with the most efficient methods relying on the subresultant algorithm (see, e.g., [1, 3, 8]). Since its original development, refinements by Lazard and Ducos have led to an improved version, commonly referred to as the *improved* subresultant algorithm [6]. The algorithm achieves a bit complexity that is quadratic in both p and the bit size of the coefficients of the input polynomials.

When R is a field, or an integral domain with an effective fraction field, the specialization property of the resultant $\mathrm{Res}(f, g) := \mathrm{sResP}_0(f, g)$ enables the use of evaluation-interpolation techniques. Building on this observation, Collins's algorithm [5] applies multi-modular methods to compute the resultant efficiently, while relying on pseudo-remainder sequences rather than subresultant polynomials.

In practice, the improved subresultant algorithm is the default method for resultant computation in FLINT. Maple also relies on subresultant-based methods, supplemented by multi-modular techniques for polynomials of high degree. However, neither software provides a direct interface for computing subresultant polynomials. In contrast, Wolfram offers explicit access to subresultant polynomials via the `SubresultantPolynomials` function.

Contributions. In this report, we design and implement a fast algorithm for computing the resultant and subresultant sequence of two multivariate polynomials over $\mathbb{Q}$, $\mathbb{Z}$, or $\mathbb{F}_p$. The approach leverages the specialization property of subresultant polynomials and can be viewed as a generalization of Collins's algorithm for computing resultants. Our implementation in FLINT demonstrates significant performance improvements over existing approaches in Wolfram, Maple, and FLINT itself, achieving speedups of up to 7x for resultant of bivariate polynomials with exponents of bitsize 7 and coefficients of bitsize 50. We discuss the algorithmic design, implementation details, performance benchmarks, and potential avenues for further optimization in the subsequent sections of this report.

2 Preliminaries

Throughout the section, we let R be a ring, and $f, g \in R[x]$ be two non-zero univariate polynomials of degree p and q, respectively. Thus, we can write $f = a_p x^p + a_{p-1}x^{p-1} + \cdots + a_0$ and $g = b_q x^q + b_{q-1}x^{q-1} + \cdots + b_0$, with $a_p, b_q \neq 0$.

2.1 Resultant

Definition 3 (Sylvester matrix)**.** *The Sylvester matrix associated to f and g is the $(p+q) \times (p+q)$ matrix defined as follows:*

$$S(f,g) = \begin{pmatrix} a_p & & & & b_q & & & \\ a_{p-1} & a_p & & & b_{q-1} & b_q & & \\ \vdots & \vdots & \ddots & & \vdots & \vdots & \ddots & \\ a_0 & a_1 & \cdots & a_p & b_0 & b_1 & \cdots & b_q \\ & a_0 & \ddots & \vdots & & b_0 & \ddots & \vdots \\ & & \ddots & a_1 & & & \ddots & b_1 \\ & & & a_0 & & & & b_0 \end{pmatrix},$$

where all the non-specified coefficients are zero.

In Sylvester's original paper [9], no explicit row or column convention was specified for the Sylvester matrix. Consequently, some sources in the literature (e.g., [2]) present the Sylvester matrix as the transpose of the form given above. In this report, we adopt a column-oriented convention, under which the Sylvester matrix is the matrix of the linear map

$$\begin{aligned} \phi : R[x]_{<q} \oplus R[x]_{<p} &\to R[x]_{<p+q} \\ (u, v) &\mapsto uf + vg \end{aligned}$$

in the canonical monomial basis $(1, x, x^2, \ldots)$. This leads to the definition of the resultant as follows, as well as the following fundamental property of the resultant.

Definition 4 (Resultant)**.** *The resultant of f and g, denoted by* $\mathrm{Res}(f,g)$*, is defined as the determinant of the Sylvester matrix $S(f,g)$.*

Proposition 5. *Suppose that R is a domain, and let $\mathbb{K}$ be its fraction field. Then,* $\mathrm{Res}(f,g) \neq 0$ *if and only if* $\gcd(f,g) = 1$ *in* $\mathbb{K}[x]$.

Proof. As $\mathrm{Res}(f,g)$ is the determinant of the linear map ϕ, we have $\mathrm{Res}(f,g) = 0 \iff \ker(\phi) \neq \{0\}$, i.e., there exist non-zero polynomials u and v, with $\deg(u) < q$ and $\deg(v) < p$, such that $uf + vg = 0$. We now show that this is equivalent to $\gcd(f,g) \neq 1$.

($\Longleftarrow$): Assume $\gcd(f,g) \neq 1$. Let $h = \gcd(f,g)$, and $f_1, g_1 \in R[x]$ such that $f = hf_1$ and $g = hg_1$. Then, we can take $u = g_1$ and $v = -f_1$, so that $uf + vg = g_1hf_1 - f_1hg_1 = 0$.

($\Longrightarrow$): Assume that there exist non-zero polynomials u and v, with $\deg(u) < q$ and $\deg(v) < p$, such that $uf + vg = 0$. We can write $u = du_1$ and $g = dg_1$, where $d = \gcd(u,g)$ and $\gcd(u_1, g_1) = 1$. Then, we have $u_1 f = -vg_1$, which implies that $g_1 \mid u_1 f$. Since $\gcd(u_1, g_1) = 1$, we deduce that $g_1 \mid f$. Thus, g_1 is a common divisor of f and g. Moreover, we have $\deg(u) < q = \deg(g)$, which implies that $\deg(d) + \deg(u_1) < \deg(d) + \deg(g_1)$, so that $\deg(u_1) < \deg(g_1)$. Hence, g_1 is not constant, and we conclude that $\gcd(f,g) \neq 1$. □

2.2 Subresultant polynomials

Likewise, we can define subresultant polynomials with the determinants of certain submatrices of the Sylvester matrix. Suppose that $p > q$.

Definition 6 (Sylvester-Habicht matrix)**.** *Given $0 \leq i \leq q$, the i-th Sylvester-Habicht matrix $S_i(f,g)$ is the $(p+q-i) \times (p+q-2i)$ submatrix of $S(f,g)$ obtained as follows: take the first $p-i$ columns of $S(f,g)$ without their last i zero rows, then append the next $q-i$ columns starting from the $(p+1)$-th row of $S(f,g)$ without their last i zero rows.*

For $0 \leq j \leq i$, we denote by $M_{i,j}(f,g)$ the $(p+q-2i) \times (p+q-2i)$ submatrix of $S_i(f,g)$ obtained by taking the first $p+q-2i-1$ rows of $S_i(f,g)$ and the $(p+q-i-j)$-th row of $S_i(f,g)$. Then, $M_{i,i}(f,g)$ is the matrix of the linear map

$$\begin{aligned} \phi_i : R[x]_{<q-i} \oplus R[x]_{<p-i} &\to R[x]_{<p+q-2i} \\ (u,v) &\mapsto uf + vg \end{aligned}$$

in the canonical monomial basis. The determinant of $M_{i,j}(f,g)$ is often referred to as the j-th subresultant $\mathrm{sRes}_j(f,g)$ of f and g. We now give the definition of the i-th subresultant polynomial of f and g.

Definition 7 (Subresultant polynomial)**.** *Given $0 \leq i \leq q$, the i-th subresultant polynomial of f and g, denoted by $\mathrm{sResP}_i(f,g)$, is defined as*

$$\mathrm{sResP}_i(f,g) = \sum_{j=0}^{i} \det(M_{i,j}(f,g)) x^j.$$

We say that $\mathrm{sResP}_i(f,g)$ is non-defective if $\deg(\mathrm{sResP}_i(f,g)) = i$ (i.e., $\mathrm{sRes}_i(f,g) \neq 0$), and defective otherwise.

We now state the structure theorem for subresultants, which gives the relation between subresultant polynomials. An immediate consequence is that non-defective subresultant polynomials are essentially the remainders in the Euclidean algorithm, up to a scaling factor.

Theorem 8 ([2, Thm. 8.56])**.** *Let $0 \leq i \leq q$. Suppose that $\mathrm{sResP}_{i-1}(f,g)$ is non-zero and of degree $0 \leq j \leq i-1$. Then,*

- *If $\mathrm{sResP}_{j-1}(f,g)$ is zero, then $\mathrm{sResP}_{i-1}(f,g) \propto \gcd(f,g)$.*

  Moreover, $\mathrm{sResP}_k(f,g) = 0$ for all $k \leq j-1$.

- *If $\mathrm{sResP}_{j-1}(f,g)$ is non-zero and of degree $0 \leq k \leq j-1$, then $\mathrm{sResP}_{k-1}(f,g)$ is proportional to $\mathrm{Rem}(\mathrm{sResP}_{i-1}(f,g), \mathrm{sResP}_{j-1}(f,g))$.*

  Moreover, if $\mathrm{sResP}_{j-1}(f,g)$ is defective, then $\mathrm{sResP}_k(f,g) \propto \mathrm{sResP}_{j-1}(f,g)$, and for all $k+1 \leq \ell \leq j-2$, $\mathrm{sResP}_\ell(f,g) = 0$.

We refer the reader to [2, Chapter 8] for a detailed proof of the structure theorem for subresultants, as well as the explicit formulas for the scaling factors. This theorem enables computation of the sequence of subresultant polynomials, and hence the GCD and resultant of two polynomials, without performing any division in the fraction field of R. The algorithm is given as follows.

Algorithm 1 The subresultant algorithm

Require: $f, g \in R[x]$, $p = \deg(f)$, $q = \deg(g)$, $p > q$
Ensure: $(\mathrm{sResP}_i(f,g))_{0 \le i \le q}$

$\mathrm{sResP}_p \leftarrow f$
$s_p \leftarrow 1$, $t_p \leftarrow 1$
$\mathrm{sResP}_{p-1} \leftarrow g$
$t_{p-1} \leftarrow \mathrm{lc}(\mathrm{sResP}_{p-1})$
$\mathrm{sResP}_q \leftarrow (-1)^{(p-q)(p-q-1)/2} \mathrm{lc}(\mathrm{sResP}_{p-1})^{p-q-1} \mathrm{sResP}_{p-1}$
$s_q \leftarrow \mathrm{lc}(\mathrm{sResP}_q)$
$i \leftarrow p+1$, $j \leftarrow p$
while $\mathrm{sResP}_{j-1} \neq 0$ **do**
    $k \leftarrow \deg(\mathrm{sResP}_{j-1})$
    // Lazard's optimization for computing $s_k = \mathrm{sRes}_k(f,g)$
    for $\delta = 1$ to $j-k-1$ **do**
        $t_{j-\delta-1} \leftarrow (-1)^{\delta}(t_{j-1}t_{j-\delta})/s_j$
    end for
    $s_k \leftarrow t_k$
    // Computation of defective subresultant polynomials (optional)
    $\mathrm{sResP}_k \leftarrow s_k \mathrm{sResP}_{j-1} / t_{j-1}$
    for $\ell = k+1$ to $j-1$ **do**
        $\mathrm{sResP}_\ell \leftarrow 0$
    end for
    // Computation of the next non-defective subresultant polynomial
    $\mathrm{sResP}_{k-1} \leftarrow -\mathrm{Rem}(s_k t_{j-1} \mathrm{sResP}_{i-1}, \mathrm{sResP}_{j-1})/(s_j t_{i-1})$
    $t_{k-1} \leftarrow \mathrm{lc}(\mathrm{sResP}_{k-1})$
    $i \leftarrow j$, $j \leftarrow k$
end while
for $\ell = 0$ to $j-2$ **do**
    $\mathrm{sResP}_\ell \leftarrow 0$
end for
return $(\mathrm{sResP}_i)_{0 \le i \le q}$

We emphasize that for the subresultant polynomials to be well-defined in $R[x]$, the assumption that $p > q$ is crucial. In the case of $p = q$, the algorithm defines $\mathrm{sResP}_q(f,g) = b_q^{-1} g$, which may fail to lie in $R[x]$ if b_q is not a unit. Nevertheless, the polynomials $\mathrm{sResP}_i(f,g)$ for $0 \le i \le q-1$ remain well-defined in $R[x]$ and can be computed by the algorithm; we do not pursue the case of $p = q$ in this report for simplicity.

Finally, notice that the above algorithm 1 can also be used to calculate the subresultant sequence, as subresultants are just leading coefficients of subresultant polynomials find by this algorithm.

3 Reduction properties

Let $n \geq 2$, and $f, g \in R[x_1, \ldots, x_n]$, where R is either $\mathbb{Z}$, $\mathbb{Q}$, or $\mathbb{F}_p$. f and g can be viewed as univariate polynomials in x_1 with coefficients in $R[x_2, \ldots, x_n]$. The resultant of f and g with respect to x_1, denoted by $\mathrm{Res}_{x_1}(f, g)$, is then defined as the resultant of these univariate polynomials in x_1; the subresultant polynomials are defined analogously.

While subresultant algorithms can be used to compute $\mathrm{Res}_{x_1}(f, g)$ (resp. the subresultants), their efficiency may be compromised by the rapid growth of the coefficients in $R[x_2, \ldots, x_n]$ during the computation. To mitigate this, one may instead evaluate the input polynomials at suitable values of $x_2, \ldots, x_n$, compute the resultant of the resulting univariate polynomials in x_1, and then interpolate to recover $\mathrm{Res}_{x_1}(f, g)$. This evaluation-interpolation approach replaces expensive symbolic computations in $R[x_2, \ldots, x_n]$ with more efficient numerical computations in R, often leading to significant performance improvements.

Still, it is crucial to ensure that the evaluation points are chosen such that the interpolation process is valid and yields the correct result. In the following, we discuss the conditions under which this approach is valid and how to choose suitable evaluation points.

3.1 Specialization property of the resultant

Intuitively, let $\eta \in R^{n-1}$, and $f(\eta), g(\eta) \in R[x_1]$ be the univariate polynomials obtained by evaluating f and g at η. We then have $\mathrm{Res}_{x_1}(f(\eta), g(\eta)) \in R$. We would like to have $\mathrm{Res}_{x_1}(f(\eta), g(\eta)) = \mathrm{Res}_{x_1}(f, g)(\eta)$, so that we can compute $\mathrm{Res}_{x_1}(f, g)$ by interpolation. However, this equality does not hold for all η:

Example 9. *Let $f = x_1x_2 + 1$ and $g = x_1 + 1$. Then, $\mathrm{Res}_{x_1}(f, g) = x_2 - 1$. However, for $\eta = 0$, we have $\mathrm{Res}_{x_1}(f(\eta), g(\eta)) = 1 \neq \mathrm{Res}_{x_1}(f, g)(\eta) = -1$.*

What happens in the above example is that the degree of f and g changes after evaluation, which leads to Sylvester matrix of different sizes. For the equality $\mathrm{Res}_{x_1}(f(\eta), g(\eta)) = \mathrm{Res}_{x_1}(f, g)(\eta)$ to hold, we should ensure that the degree of f and g does not change after evaluation. Formally, we have the following result.

Proposition 10. *Let $f, g \in R[x_1, \ldots, x_n]$, $\eta \in R^{n-1}$, and $h = \mathrm{lc}_{x_1}(f)\,\mathrm{lc}_{x_1}(g) \in R[x_2, \ldots, x_n]$ be the product of the leading coefficients of f and g with respect to x_1. If $h(\eta) \neq 0$, then $\mathrm{Res}_{x_1}(f(\eta), g(\eta)) = \mathrm{Res}_{x_1}(f, g)(\eta)$.*

Proof. $h(\eta) \neq 0$ implies that $\mathrm{lc}_{x_1}(f)(\eta) \neq 0$ and $\mathrm{lc}_{x_1}(g)(\eta) \neq 0$. Therefore, $\deg_{x_1}(f(\eta)) = \deg_{x_1}(f)$ and $\deg_{x_1}(g(\eta)) = \deg_{x_1}(g)$, so that $S(f(\eta), g(\eta)) = S(f, g)(\eta)$, hence the result. □

When R is either $\mathbb{Z}$ or $\mathbb{Q}$, the property that $h \neq 0$ is a generic condition, as $\{\eta \in \mathbb{C}^{n-1} : h(\eta) = 0\}$ is a proper Zariski-closed subset of $\mathbb{C}^{n-1}$. Consequently, we can find sufficiently many evaluation points as we want such that h does not vanish at these points, and then interpolate to recover $\mathrm{Res}_{x_1}(f, g)$. However, it is not the case when $R = \mathbb{F}_p$ for whatever p. A natural question is to ask how many evaluation points we need to recover $\mathrm{Res}_{x_1}(f, g)$ by interpolation. This is answered by the following degree bound of the resultant.

Proposition 11. *Let $f, g \in \mathbb{Z}[x_1, \ldots, x_n]$. Denote by $\deg_{x_2,\ldots,x_n}(f)$ (resp. $\deg_{x_2,\ldots,x_n}(g)$) the total degree of f (resp. g) with respect to $x_2, \ldots, x_n$. Then, we have*

$$\deg(\mathrm{Res}_{x_1}(f, g)) \leq D := \deg_{x_1}(f)\deg_{x_2,\ldots,x_n}(g) + \deg_{x_1}(g)\deg_{x_2,\ldots,x_n}(f).$$

Proof. Each term in $\det(S(f, g))$ is a product of $\deg_{x_1}(g)$ coefficients of f and $\deg_{x_1}(f)$ coefficients of g, and therefore has degree bounded by D. We can then conclude by the definition of the resultant as the determinant of the Sylvester matrix. □

Hence, we can recover $\mathrm{Res}_{x_1}(f,g)$ by multivariate interpolation from at most $\binom{D+n-1}{n-1}$ evaluation points, thus from at most $D+1$ evaluation points when $n=2$. When $R=\mathbb{F}_p$, h can at most vanish at $p^{n-2}\deg(h)$ points, so that when p is large enough such that $p^{n-2}(p-\deg(h)) > \binom{D+n-1}{n-1}$, we can still recover $\mathrm{Res}_{x_1}(f,g)$ by interpolation.

3.2 Specialization property of subresultant polynomials

For the subresultant polynomials, as each coefficient of $\mathrm{sResP}_i(f,g)$ is the determinant of a submatrix of the Sylvester matrix, we have the same specialization property as the resultant, and the same condition on the evaluation points.

Proposition 12. *Let $f,g \in \mathbb{Z}[x_1,\ldots,x_n]$ with $\deg_{x_1}(f) > \deg_{x_1}(g) \geq 0$, $\eta \in \mathbb{Z}^{n-1}$, and $h = \mathrm{lc}_{x_1}(f)\,\mathrm{lc}_{x_1}(g) \in \mathbb{Z}[x_2,\ldots,x_n]$. If $h(\eta) \neq 0$, then $\mathrm{sResP}_i(f(\eta),g(\eta)) = \mathrm{sResP}_i(f,g)(\eta)$ for all $0 \leq i \leq \deg_{x_1}(g)$.*

Proof. We already know that when $h(\eta)\neq 0$, we have $S(f(\eta),g(\eta)) = S(f,g)(\eta)$. Hence, we have $M_{i,j}(f(\eta),g(\eta)) = M_{i,j}(f,g)(\eta)$ for all $0 \leq j \leq i \leq \deg_{x_1}(g)$, so that $\mathrm{sResP}_i(f(\eta),g(\eta)) = \mathrm{sResP}_i(f,g)(\eta)$ for all $0 \leq i \leq \deg_{x_1}(g)$. □

To compute $\mathrm{sResP}_i(f,g)$, we can then compute $\mathrm{sResP}_i(f(\eta),g(\eta))$ for sufficiently many evaluation points η, and then interpolate to recover each coefficient of $\mathrm{sResP}_i(f,g)$. The number of evaluation points needed is bounded by the same bound as the resultant, as each coefficient of $\mathrm{sResP}_i(f,g)$ is a determinant of a submatrix of the Sylvester matrix, and therefore has degree bounded by D.

Computing non-defective subresultant polynomials. Algorithm 1 can be readily modified to compute only the non-defective subresultant polynomials by omitting Lines 15-19. One immediate application of this variant is the computation of the Sturm–Habicht sequence of f, i.e., the sequence of non-defective subresultant polynomials of f and its derivative $\frac{\partial f}{\partial x_1}$, up to sign changes.

A natural question is to ask whether we can compute non-defective subresultant polynomials more efficiently than computing all subresultant polynomials. The answer is positive, but one needs to take extra attention to the choice of evaluation points.

Example 13. *Let $f = x_1^4 + x_1^2x_2 + 1$ and $g = x_1^3 + x_1 + x_2$. Then, the non-defective subresultant polynomials of f and g with respect to x_1 are*

$$\mathrm{sResP}_0(f,g) = x_2^5 - x_2^4 - 2x_2^3 + 5x_2^2 - 4x_2 + 4, \quad \mathrm{sResP}_1(f,g) = (2x_2^2 - 3x_2 + 2)x_1 + x_2^3 - 2x_2^2,$$
$$\mathrm{sResP}_2(f,g) = (x_2-1)x_1^2 - x_2x_1 + 1, \qquad \mathrm{sResP}_3(f,g) = x_1^3 + x_1 + x_2.$$

However, $\mathrm{sResP}_2(f,g)$ becomes defective after evaluating at $x_2 = 1$.

Hence, the good evaluation points for computing non-defective subresultant polynomials are those that do not make any non-defective subresultant polynomial defective.

While this condition is likewise generic, it is less straightforward to verify than the condition for the resultant, where it suffices to check whether $\mathrm{lc}_{x_1}(f)\,\mathrm{lc}_{x_1}(g)$ vanishes at the evaluation points. Nevertheless, both the number of non-defective subresultant polynomials and their degrees can be determined by evaluating at a sufficiently large set of random points and inspecting the degree of the resulting subresultant polynomials. One can then select evaluation points for which the observed degree sequence matches the expected one, and finally reconstruct the polynomials via interpolation.

4 Proposed Algorithm

In this section, we propose an algorithm to efficiently compute the resultant of two bivariate polynomials, f and g in $\mathbb{Z}$ using an evaluation-interpolation approach. First, we recall the results from the previous section:

1. The resultant of two evaluated polynomials is generically equal to the resultant evaluated at the same point, outside the set of points where either $\mathrm{lc}_{x_2}(f)$ or $\mathrm{lc}_{x_2}(g)$ vanishes.
2. The resultant of two bivariate polynomials has a degree bounded by $\deg_{x_1}(f)\deg_{x_2}(g) + \deg_{x_1}(g)\deg_{x_2}(f)$.

Using these two facts, we propose the following algorithm.

Algorithm 2 BivariateResultant

Require: $f, g \in \mathbb{Z}[x]^2$
Ensure: $\mathrm{Res}_{x_2}(f, g) \in \mathbb{Z}[x_1]$
$N \leftarrow \deg_{x_1}(f) \times \deg_{x_2}(g) + \deg_{x_1}(g) \times \deg_{x_2}(f)$
// We choose evaluation points starting from 1 and increasing,
// until we have $N + 1$ points where leading coefficients do not vanish
$k \leftarrow 1$
for $i = 1$ to $N + 1$ **do**
    $x_i \leftarrow k$
    while $\mathrm{lc}_{x_2}(f)(x_i) = 0$ or $\mathrm{lc}_{x_2}(g)(x_i) = 0$ **do**
        $k \leftarrow k + 1$
        $x_i \leftarrow k$
    end while
    $k \leftarrow k + 1$
end for
// We evaluate polynomials on chosen points
for $i = 1$ to $N + 1$ **do**
    This resultant is in $\mathbb{Z}$ since polynomials are evaluated.
    $y_i \leftarrow \mathrm{Res}(f(x_i), g(x_i))$
end for
// We interpolate the resultant in $\mathbb{Z}[x_1]$
$R \leftarrow \mathsf{interpolate}(x, y, N)$
return R

Note that interpolation points can also be chosen with another method, e.g. randomly on a given bitsize or in the order of increasing bitsize. For $\mathbb{Z}$, a good choice is to choose points in the order $0, 1, -1, 2, -2, \ldots$, etc. We implement these different approaches in our work and present their differences on performance in the following sections.

It is also important to note that the above algorithm can be generalized to subresultants or subresultant polynomials. In our work, we also present an implementation of this approach for subresultants, using again our own implementation for univariate polynomial subresultant computation, as FLINT does not expose the subresultant algorithm. We do not include the pseudocode of the algorithm, as it is almost the same as the resultant computation, with the slight difference of repeating the same process for each subresultant instead of a single time.

5 Benchmarks

Here, we present some benchmarkings for the comparison of our algorithm against the FLINT library for bivariate polynomial resultant computation.

All benchmarks presented below are performed on an Apple MacBook with 8-core Apple M2 SoC, 16 GB unified memory. All results are the average running time of 5 independent trials, and the bitsize of polynomial coefficients is fixed to be 50 in all cases.

Note that for our proposed implementation, we present 6 different running times, based on the selection method of points that are used to interpolate the polynomial:

1. Random Points: Fully random points in the bitsize of polynomial coefficients.
2. Random Positive Points: Fully random and positive points in the bitsize of polynomial coefficients.
3. Small Random Points: Small random points in the bitsize of the degree of the polynomial.
4. Small Random Positive Points: Small random and positive points in the bitsize of the degree of the polynomial.
5. Ordered Points: Ordered points in the ascending order of bitsize.
6. Ordered Positive Points: Ordered and positive points in the ascending order of bitsize.

5.1 FMPZ Benchmarks

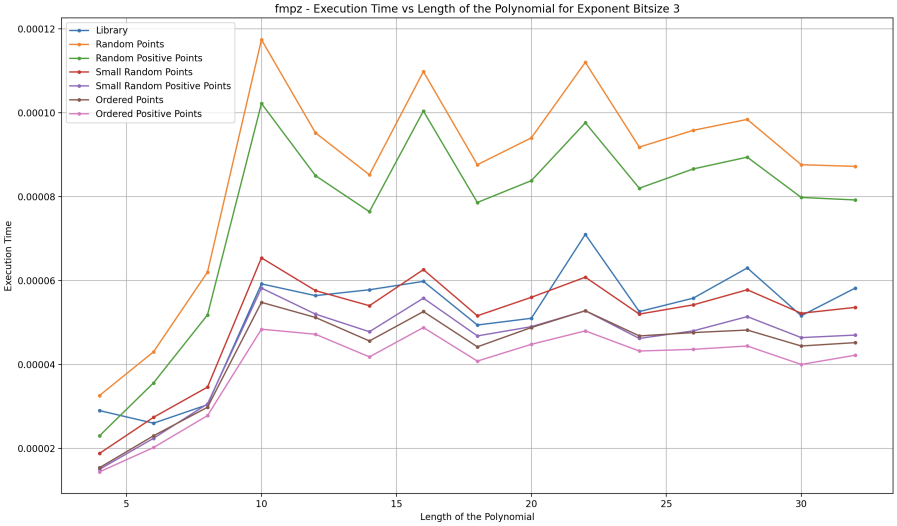

Figure 1: FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 3 with varying lengths.

In Figure 1, we first compare our implementation against the library in small polynomials, i.e. of exponent bitsize 3. We see that while the random selection of points for the interpolation method is significantly slower than the rest, other methods have similar running times. This important difference between random selection and others can be explained by the bitsize of the points used in the interpolation. When a uniformly random selection is performed, bitsize of the points used in the interpolation is the same as the bitsize of coefficients, i.e. 50. Thus, the interpolation takes significantly more time than other approaches.

Another important observation, about rest of the measured methods, is that even though FLINT's implementation is slower in some cases, there is no important difference between their running times.

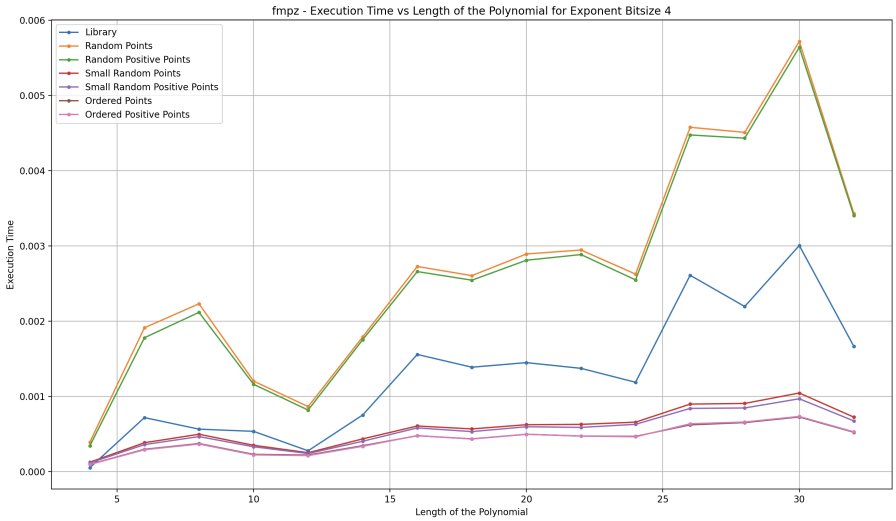

Figure 2: FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 4 with varying lengths.

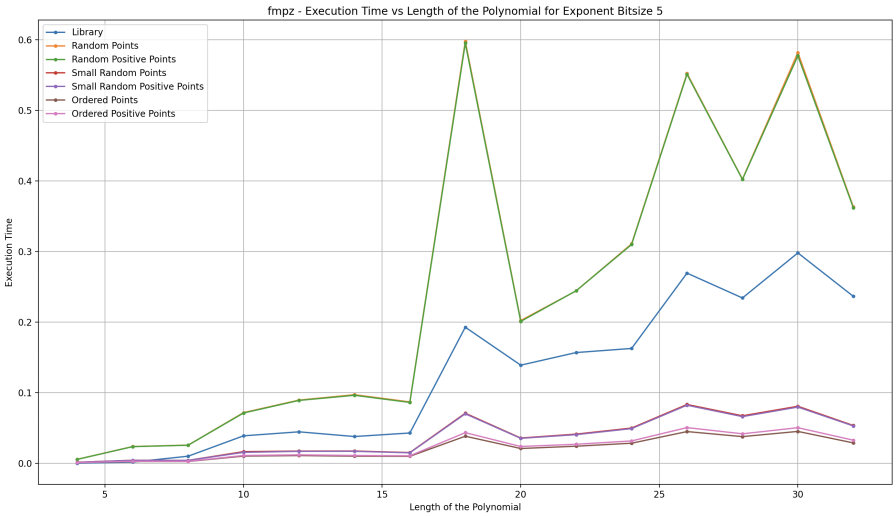

Figure 3: FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 5 with varying lengths.

In Figure 2 and 3, we extend our analysis to bigger polynomials with the exponent bitsize 4 and 5, respectively. We see that the relation between the random selection method and other categories is still the same, while the difference between the FLINT library implementation and our methods starts to become visible. As the length of polynomials increase, FLINT performs much slower than our algorithm. Notice also that there is not too much difference between randomly selecting interpolation points with small bitsize and having them ordered in the bitsize, which supports our intiution that the running time of polynomial interpolation depends on the bitsize of selected points.

Moreover, we can observe that all curves follow a similar shape with considerable fluctuations. Even though the overall trend is that increasing the polynomial's length increases the running time, we see that for some lengths there is a sharp decrease in the measure time. This

result can be explained by the properties of the random bivariate polynomial generation function of FLINT. For some testcases, the generated polynomials have a resultant that is much harder to compute. For instance, when we generate random bivariate polynomials in FLINT with parameters of length 10, exponent bitsize 5, and coefficient bitsize 8, we may get the following two polynomials:

$$f_1 = 3x^5 + x^3y^5 - 63x^2y + xy^3 - 14y^5 + 11y$$

$$f_2 = x^{15}y^{11} + x^{15} + 45x^{12}y^8 + 55x^4y + 3x^3y + 226x^2 - 55xy^4 + 9x + 31$$

Both of these two polynomials are generated by the exact same function call, but they differ a lot in terms of arithmetic operations while computing their resultant. Thus, the random polynomial generation algorithm of FLINT cannot be efficiently used to have a more stable benchmark. A better random polynomial generation algorithm can theoretically be used, which we provide an example of for univariate polynomials in our work. We do not extend our simple implementation to random generation of bivariate polynomials as it falls way out of scope of this paper.

Moreover, these fluctuations can also be minimized by increasing number of trials per length. Yet, increasing number of trials also increase the total benchmarking time significantly, and since our main motivation is comparing different methods, but not giving an accurate estimate of their dependence of the length of the polynomial, we keep this methodology.

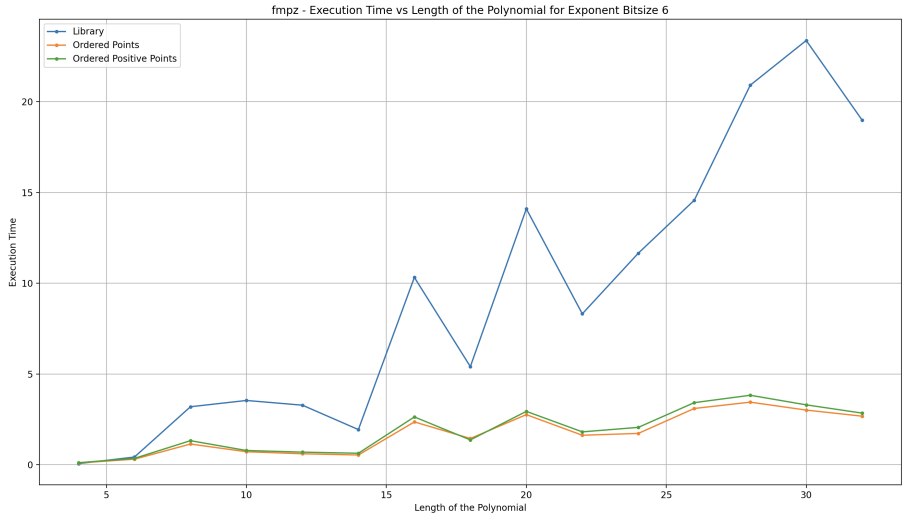

Figure 4: FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 6 with varying lengths.

Finally, we measure with polynomials of the exponent bitsize 6, where we only include the library and ordered implementations to minimize the total running time. We observe that the difference between both methods becomes even more important, where we achieve an approximate of 7× performance increase compared to FLINT. For bigger bitsizes, total running times exceeds 30 seconds for a single measurement, thus we do not present the results here.

5.2 FMPQ Benchmarks

In this section, we extend our analysis to polynomials over $\mathbb{Q}$ instead of $\mathbb{Z}$, but note that presented results do not really differ from what we have observed for $\mathbb{Z}$. Interpolation using random points with big bitsize is significantly slower than rest of the methods, and the FLINT implementation starts becoming much slower than our implementations as the exponent bitsize

and the length of the polynomials increase. Using ordered points in bitsize for interpolation is slightly more performant than using random points with small bitsize, but both methods are still significantly faster than FLINT. The maximum achieved performance improvement is again around 7× compared to FLINT.

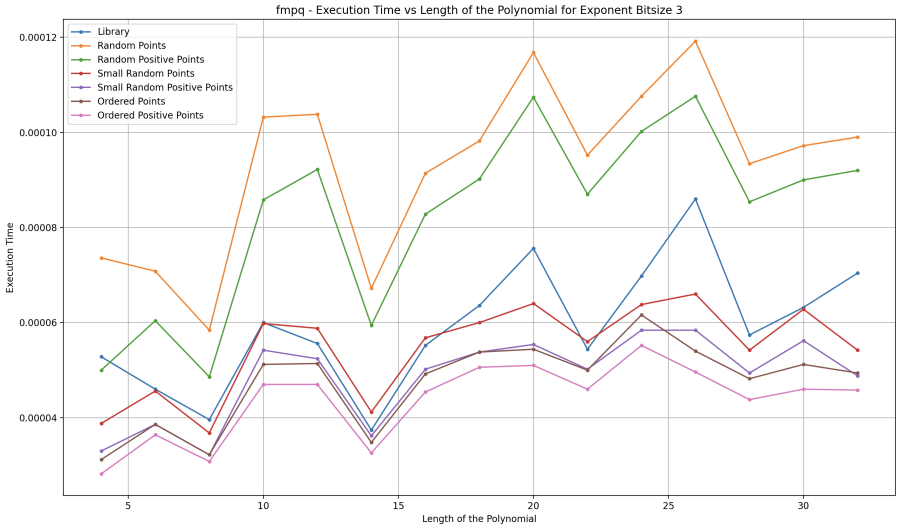

Figure 5: FMPQ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 3 with varying lengths.

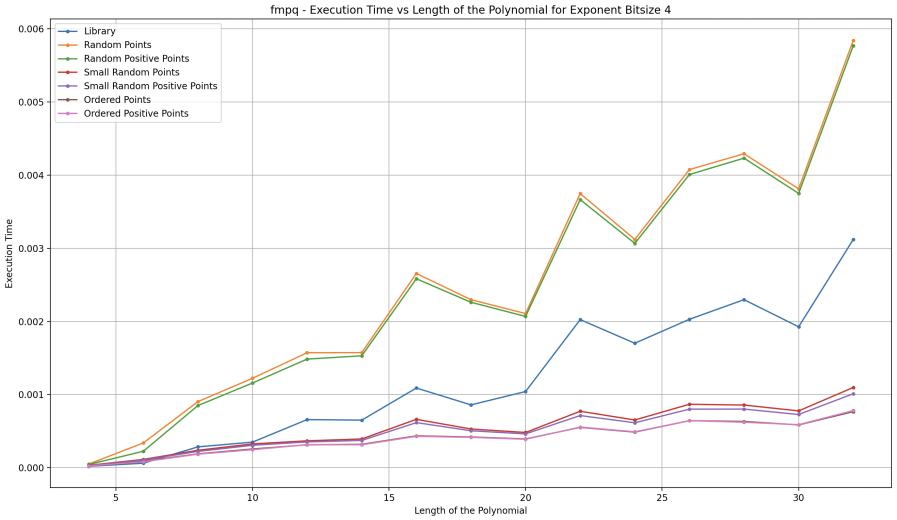

Figure 6: FMPQ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 4 with varying lengths.

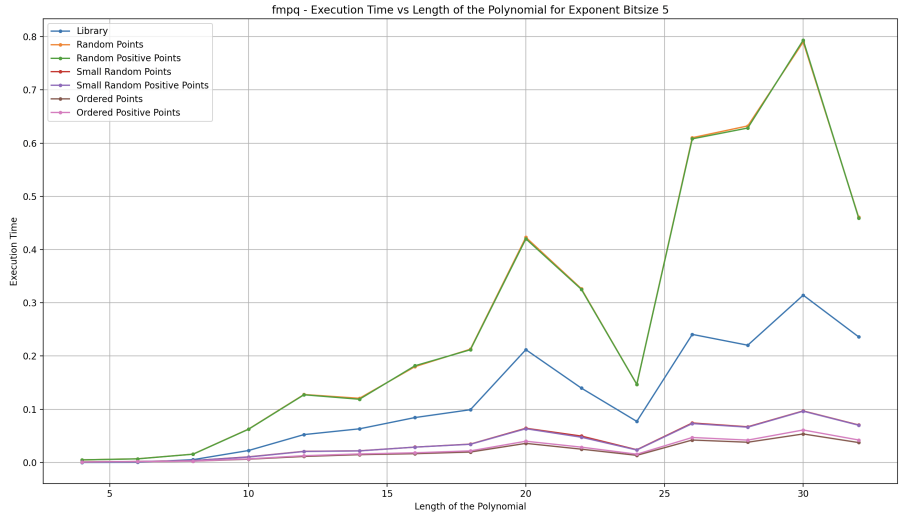

Figure 7: FMPQ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 5 with varying lengths.

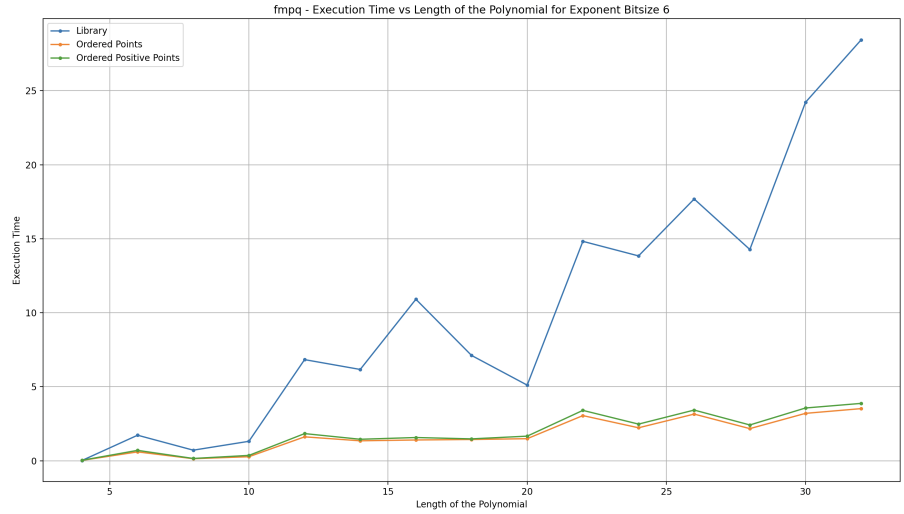

Figure 8: FMPQ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 6 with varying lengths.

Finally, we note that even though we implement and test our algorithm for the subresultant sequence of polynomials as well, we do not present any benchmarks here, since FLINT does not expose subresultant or subresultant polynomial sequences, so it is not possible to give any comparison.

6 Conclusion

We conclude that using an evoluation-interpolation approach for computing resultant, subresultants or subresultant polynomials may perform significantly faster than existing methods in well-known libraries such as FLINT. We observe a 7× performance improvement in polynomials with exponent bitsize 7 and coefficient bitsize 50. We also note that the performance difference becomes more significant as the degree and length of polynomials increase.

Moreover, we also observe that the method used to choose random points for the interpolation step of our algorithm affects the performance heavily. In particular, the interpolation performs faster when working with points of smaller bitsize. Thus, we identify the fastest the

implementation as the one that choses points ordered in increasing bitsize instead of using any randomization.

As possible next steps of our work, we identify:

- Benchmarking the subresultant implementation for bivariate polynomials (by choosing Wolfram or similar as the reference point),
- Extending the implementation from resultants / subresultants to subresultant polynomials,
- Extending the implementation to higher dimension polynomials, e.g. trivariate,
- Optimize, test, and benchmark for higher degree and length polynomials.

References

[1] Alkiviadis Akritas. *Elements of computer algebra with applications.* John Wiley & Sons, Inc., 1989.

[2] Saugata Basu, Richard Pollack, and Marie-Françoise Roy. *Algorithms in real algebraic geometry.* Springer, 2006.

[3] Henri Cohen. *A course in computational algebraic number theory.* Springer Science & Business Media, 2013.

[4] George E Collins. Subresultants and reduced polynomial remainder sequences. *Journal of the ACM (JACM)*, 14(1):128–142, 1967.

[5] George E Collins. The calculation of multivariate polynomial resultants. *Journal of the ACM (JACM)*, 18(4):515–532, 1971.

[6] Lionel Ducos. Optimizations of the subresultant algorithm. *Journal of Pure and Applied Algebra*, 145(2):149–163, 2000.

[7] Leonhard Euler. Demonstration sur le nombre des points, ou deux lignes des ordres quelconques peuvent se couper. *Memoires de l'academie des sciences de Berlin*, pages 234–248, 1750.

[8] Keith O Geddes, Stephen R Czapor, and George Labahn. *Algorithms for computer algebra.* Springer, 1992.

[9] James Joseph Sylvester. Xviii. on a theory of the syzygetic relations of two rational integral functions, comprising an application to the theory of sturm's functions, and that of the greatest algebraical common measure. *Philosophical transactions of the Royal Society of London*, (143):407–548, 1853.